VTYS PROJE RAPORU

WalkTogether

Topluluk Egzersiz ve Yürüyüş Rotası Oluşturma Sistemi

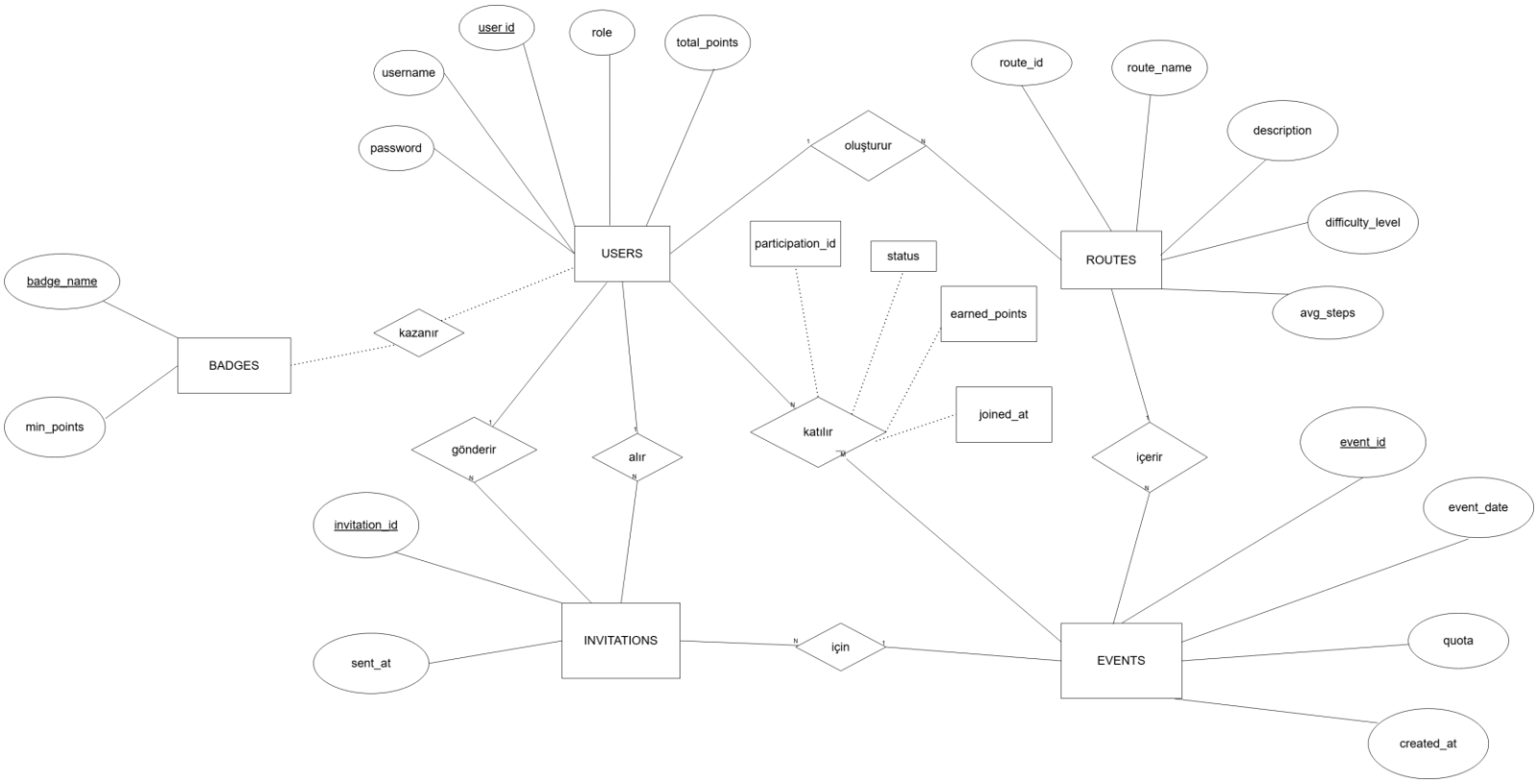
HAZIRLAYANLAR

Berkay Bolat - 22011043

Emirhan Coşkun - 22011045

Oytun Utkan Yeşilyurt - 23011014

1. Tasarım (ER Diyagramı)



2. Tablolar (Ekran Görüntüleri)

“users” Tablosu

	user_id [PK] integer	username character varying (50)	password character varying (50)	role character varying (20)	total_points integer
1	1	Berkay	1234	admin	10000
2	2	Oytun	1234	admin	10000
3	3	Emirhan	1234	admin	7000
4	4	Hasan	1234	user	0
5	5	Mehmet	1234	user	100
6	6	Ahmet	1234	user	600
7	7	Fatih	1234	user	4200
8	8	Emre	1234	user	1300
9	9	Dila	1234	user	700
10	10	Elif	1234	user	500

“routes” Tablosu

	route_id [PK] integer	route_name character varying (100)	difficulty_level integer	avg_steps integer	description text	creator_id integer
1	1	Sahil Yürüyüşü	1	3000	Bebek sahil boyunca hafif tempo yürüyüş.	1
2	2	Orman Yürüyüşü	3	4000	Doğa ile iç içe hızlı tempoda yürüyüş.	3
3	3	Şehir Merkezi Turu	2	5000	Şehir merkezinde, caddelerde yürüyüş.	1
4	4	Dağ Tırmanışı	5	10000	Yüksek eğimli zorlu yürüyüş parkuru.	2
5	5	Göl Kenarı Koşusu	3	4500	Düz zemin koşu parkuru.	3
6	6	Köy Yürüyüşü	1	2500	Toprak yol, sessiz rota.	2
7	7	Kanyon Geçişi	4	9000	Kayalık ve engebeli zemin.	1
8	8	Gece Yürüyüşü	2	4000	Aydınlatılmış güvenli parkurda orta tempo yürüyüş.	2
9	9	Kampüs Turu	3	7000	Üniversite kampüsü içerisinde sohbet eşliğinde yürüy...	3
10	10	Maraton Hazırlık Koşusu	5	15000	Profesyoneller için uzun mesafe zorlu koşu parkuru.	2

“events” Tablosu

	event_id [PK] integer	route_id integer	event_date timestamp without time zone	quota integer	created_at timestamp without time zone
1	1	1	2026-01-20 09:00:00	20	2026-01-16 17:15:08.329316
2	2	2	2026-01-21 08:30:00	15	2026-01-16 17:15:08.329316
3	3	3	2026-01-22 12:00:00	30	2026-01-16 17:15:08.329316
4	4	4	2026-01-23 14:00:00	2	2026-01-16 17:15:08.329316
5	5	5	2026-01-24 17:00:00	25	2026-01-16 17:15:08.329316
6	6	6	2026-01-25 07:00:00	50	2026-01-16 17:15:08.329316
7	7	1	2026-01-26 09:00:00	20	2026-01-16 17:15:08.329316
8	8	2	2026-01-27 10:00:00	15	2026-01-16 17:15:08.329316
9	9	7	2026-01-28 06:00:00	5	2026-01-16 17:15:08.329316
10	10	8	2026-01-29 20:00:00	40	2026-01-16 17:15:08.329316

“event_participants” Tablosu

	participation_id [PK] integer	user_id integer	event_id integer	status character varying (20)	earned_points integer	joined_at timestamp without time zone
1	1	1	1	Kayıt Yapıldı	0	2026-01-16 17:15:08.329316
2	2	2	4	Kayıt Yapıldı	0	2026-01-16 17:15:08.329316
3	3	3	2	Kayıt Yapıldı	0	2026-01-16 17:15:08.329316
4	4	6	3	İptal	0	2026-01-16 17:15:08.329316
5	5	7	2	Kayıt Yapıldı	0	2026-01-16 17:15:08.329316
6	6	7	4	Kayıt Yapıldı	0	2026-01-16 17:15:08.329316
7	7	8	5	Kayıt Yapıldı	0	2026-01-16 17:15:08.329316
8	8	9	1	Tamamlandı	0	2026-01-16 17:15:08.329316
9	9	10	6	Kayıt Yapıldı	0	2026-01-16 17:15:08.329316
10	10	4	2	Kayıt Yapıldı	0	2026-01-16 17:15:08.329316

“badges” Tablosu

	badge_name [PK] character varying (50)	min_points integer
1	Altın	2000
2	Bronz	300
3	Elmas	10000
4	Gümüş	800
5	Platin	5000

“invitations” Tablosu

	invitation_id [PK] integer	sender_id integer	receiver_id integer	event_id integer	sent_at timestamp without time zone

3. İsteklerin Sağlandığını Gösteren Kod Blokları (Ekran Görüntüleri)

Proje ödev dosyasında belirtilen tüm madde numaraları için oluşturulan kod bloklarının ekran görüntüleri aşağıda sırası ile verilmiştir.

1. ve 2. Madde: Tablo Yapıları, Primary Key ve Foreign Key Kısıtları

```
-- ETKİNLİK KATILIMCILARI TABLOSU --
CREATE TABLE event_participants (
    participation_id INT DEFAULT nextval('participant_seq') PRIMARY KEY,                                  -- Katılım ID'si
    user_id INT NOT NULL,                                                                                  -- Kullanıcı ID'si
    event_id INT NOT NULL,                                                                                 -- Etkinlik ID'si
    status VARCHAR(20) DEFAULT 'Kayıt Yapıldı' CHECK (status IN ('Kayıt Yapıldı', 'İptal', 'Tamamlandı')), -- Katılımcı Durum Bilgisi
    earned_points INT DEFAULT 0,                                                                           -- Kazanılan Puan
    joined_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,                                                         -- Katılım Tarihi
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE,                                     -- Foreign Key
    FOREIGN KEY (event_id) REFERENCES events(event_id) ON DELETE CASCADE,                                  -- Foreign Key
    UNIQUE (user_id, event_id)                                                                             -- Benzersiz Kullanıcı
);

-- ROZET TABLOSU --
CREATE TABLE badges (
    badge_name VARCHAR(50) PRIMARY KEY,
    min_points INT NOT NULL UNIQUE
);
```

```
-- KULLANICILAR TABLOSU --
CREATE TABLE users (
    user_id INT DEFAULT nextval('user_seq') PRIMARY KEY,         -- ID
    username VARCHAR(50) UNIQUE NOT NULL,                        -- Kullanıcı İsmi
    password VARCHAR(50) NOT NULL,                               -- Şifre
    role VARCHAR(20) NOT NULL CHECK (role IN ('admin', 'user')), -- Kullanıcı ROlü
    total_points INT DEFAULT 0                                   -- Kullanıcı Puanı
);

-- ROTALAR TABLOSU --
CREATE TABLE routes (
    route_id INT DEFAULT nextval('route_seq') PRIMARY KEY,                -- Rota ID'si
    route_name VARCHAR(100) NOT NULL,                                     -- Rota İsmi
    difficulty_level INT CHECK (difficulty_level BETWEEN 1 AND 5),        -- Zorluk Seviyesi (1 -> Kolay, 5 -> Zor)
    avg_steps INT NOT NULL,                                               -- Ortalama Adım
    description TEXT,                                                     -- Rota Açıklaması
    creator_id INT,                                                       -- Rotayı Ekleyen Kullanıcı ID'si
    FOREIGN KEY (creator_id) REFERENCES users(user_id) ON DELETE SET NULL -- Foreign Key
);
```

```sql
-- DAVETLER TABLOSU --
CREATE TABLE invitations (
    invitation_id SERIAL PRIMARY KEY,
    sender_id INT NOT NULL,
    receiver_id INT NOT NULL,
    event_id INT NOT NULL,
    sent_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (sender_id) REFERENCES users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (receiver_id) REFERENCES users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (event_id) REFERENCES events(event_id) ON DELETE CASCADE,
    UNIQUE(sender_id, receiver_id, event_id)
);
```

3. Madde: Silme Kısıtı ve Sayı Kısıtı

```sql
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (event_id) REFERENCES events(event_id) ON DELETE CASCADE,
```

```sql
    FOREIGN KEY (sender_id) REFERENCES users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (receiver_id) REFERENCES users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (event_id) REFERENCES events(event_id) ON DELETE CASCADE,
```

```sql
    difficulty_level INT CHECK (difficulty_level BETWEEN 1 AND 5),
```

```sql
    CONSTRAINT check_quota_positive CHECK (quota > 0)
);
```

4. Madde: Arayüz Üzerinden INSERT, UPDATE, DELETE İşlemleri

```sql
INSERT INTO users (username, password, role, total_points) VALUES
    ('Berkay', '1234', 'admin', 10000),
    ('Oytun', '1234', 'admin', 10000),
    ('Emirhan', '1234', 'admin', 7000),
    ('Hasan', '1234', 'user', 0),
    ('Mehmet', '1234', 'user', 100),
    ('Ahmet', '1234', 'user', 600),
    ('Fatih', '1234', 'user', 4200),
    ('Emre', '1234', 'user', 1300),
    ('Dila', '1234', 'user', 700),
    ('Elif', '1234', 'user', 500);
```

```sql
        UPDATE users SET total_points = total_points + points_to_award
        WHERE user_id = NEW.user_id;

        RAISE NOTICE 'Tebrikler! Etkinliği tamamladınız ve % puan kazandınız.', points_to_award;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_award_points
AFTER UPDATE ON event_participants
FOR EACH ROW
EXECUTE FUNCTION trg_award_points_func();
```

```java
String sql = "DELETE FROM routes WHERE route_id = ?";
PreparedStatement ps = conn.prepareStatement(sql);
ps.setInt( parameterIndex: 1, routeId);
ps.executeUpdate();
```

5. Madde: Arayüz Üzerinden Girilen Bir Değere Göre Sonuç Listeleme

```java
if (discoveryMode) {

    sql.append("SELECT * FROM get_unjoined_routes(?)");
    boolean hasWhere = false;

    if (!searchText.isEmpty()) {

        sql.append(" WHERE route_name ILIKE ?");
        hasWhere = true;
    }

    if (filterHard) {

        sql.append(hasWhere ? " AND " : " WHERE ");
        sql.append("difficulty_level >= 3");
    }

    sql.append(" ORDER BY route_id");
```

6. Madde: View Tanımlaması

```sql
-- KULLANICILARI PUANLARINA GÖRE SIRALAYAN LİDERLİK TABLOSU --
CREATE OR REPLACE VIEW leaderboard_view AS
SELECT RANK() OVER (ORDER BY total_points DESC, username ASC) as rank_no,                    -- Derece Sırası
                                                                 username,                    -- Kullanıcı Adı
                                                                 role,                        -- Kullanıcı Rolü
                                                                 total_points,                -- Toplam Puan
                                                                 get_user_badge(total_points) as badge_name -- Rozet Bilgisi
FROM users;
```

```java
String sql = "SELECT * FROM leaderboard_view";
```

7. Madde: Index Kullanımı

```sql
-- UYGULAMADA ROTA İSMİNE GÖRE ARAMA YAPARKEN KULLANILACAK INDEX --
CREATE INDEX idx_route_name ON routes(route_name);
```

```java
sql.append("SELECT route_id, route_name, difficulty_level, avg_steps, description FROM routes ");
boolean hasWhere = false;

if (!searchText.isEmpty()) {

    sql.append("WHERE route_name ILIKE ? ");
    hasWhere = true;
}

if (filterHard) {

    sql.append(hasWhere ? " AND " : " WHERE ");
    sql.append("difficulty_level >= 3 ");
}

sql.append("ORDER BY route_id");
ps = conn.prepareStatement(sql.toString());
```

8. Madde: Sequence Kullanımı

```sql
-- KULLANICI ID'LERİ İÇİN OTOMATİK ARTAN SAYAÇ SEQUENCE --
CREATE SEQUENCE user_seq
    START WITH 1
    INCREMENT BY 1;

-- ROTA ID'LERİ İÇİN OTOMATİK ARTAN SAYAÇ SEQUENCE --
CREATE SEQUENCE route_seq
    START WITH 1
    INCREMENT BY 1;

-- ETKİNLİK ID'LERİ İÇİN OTOMATİK ARTAN SAYAÇ SEQUENCE --
CREATE SEQUENCE event_seq
    START WITH 1
    INCREMENT BY 1;
```

```sql
user_id INT DEFAULT nextval('user_seq') PRIMARY KEY,          -- ID
```

```sql
route_id INT DEFAULT nextval('route_seq') PRIMARY KEY,          -- Rota ID'si
```

9. Madde: UNION, INTERSECT veya EXCEPT Kullanımı

```
CREATE OR REPLACE FUNCTION get_unjoined_routes(p_user_id INT)
RETURNS TABLE (
    route_id INT,
    route_name VARCHAR,
    difficulty_level INT,
    avg_steps INT,
    description TEXT
) AS $$
BEGIN
    RETURN QUERY
    SELECT r.route_id, r.route_name, r.difficulty_level, r.avg_steps, r.description
    FROM routes r
    EXCEPT
    SELECT r.route_id, r.route_name, r.difficulty_level, r.avg_steps, r.description
    FROM routes r
    JOIN events e ON r.route_id = e.route_id
    JOIN event_participants ep ON e.event_id = ep.event_id
    WHERE ep.user_id = p_user_id;
END;
$$ LANGUAGE plpgsql;
```

10. Madde: Aggregate Fonksiyon ve HAVING Kullanımı

```
CREATE OR REPLACE FUNCTION get_admin_stats()
RETURNS TEXT AS $$
DECLARE
    pop_route VARCHAR;
    active_user VARCHAR;
    result_text TEXT;
BEGIN
    SELECT r.route_name INTO pop_route
    FROM event_participants ep
    JOIN events e ON ep.event_id = e.event_id
    JOIN routes r ON e.route_id = r.route_id
    WHERE ep.joined_at > NOW() - INTERVAL '7 days'
    GROUP BY r.route_name
    HAVING COUNT(*) > 0
    ORDER BY COUNT(*) DESC
    LIMIT 1;

    SELECT u.username INTO active_user
    FROM event_participants ep
    JOIN users u ON ep.user_id = u.user_id
    WHERE ep.joined_at > NOW() - INTERVAL '7 days'
    GROUP BY u.username
    HAVING COUNT(*) > 0
    ORDER BY COUNT(*) DESC
    LIMIT 1;

    IF pop_route IS NULL THEN pop_route := 'Henüz veri yok'; END IF;
    IF active_user IS NULL THEN active_user := 'Henüz veri yok'; END IF;

    result_text := 'Haftanın Rotası:  ' || pop_route || '      |      Haftanın Üyesi:  ' || active_user;
```

11. Madde: SQL Fonksiyonları ve CURSOR-RECORD Kullanımı

```sql
-- KULLANICI AKTİVİTE RAPORUNU GÖSTEREN FONKSİYON --
CREATE OR REPLACE FUNCTION get_user_activity_report(p_user_id INT)
RETURNS TEXT AS $$
DECLARE
    rec RECORD;
    temp_username VARCHAR;
    cur_activities CURSOR FOR
        SELECT r.route_name, e.event_id, e.event_date, ep.status
        FROM event_participants ep
        JOIN events e ON ep.event_id = e.event_id
        JOIN routes r ON e.route_id = r.route_id
        WHERE ep.user_id = p_user_id
        ORDER BY e.event_date DESC;

    report_text TEXT;
BEGIN

    SELECT username INTO temp_username FROM users WHERE user_id = p_user_id;

    report_text := '----- "' || temp_username || '" KULLANICISININ AKTİVİTE RAPORU -----' || E'\n\n';

    OPEN cur_activities;

    LOOP
        FETCH cur_activities INTO rec;
        EXIT WHEN NOT FOUND;

        report_text := report_text || 'Etkinlik ID: ' || rec.event_id ||
                                      '  ||  Rota: ' || rec.route_name ||
                                      '  ||  Tarih: ' || rec.event_date ||
                                      '  ||  Durum: ' || rec.status || E'\n\n' ||
                                      '-----------------------------------------------------' || E'\n';
    END LOOP;

    CLOSE cur_activities;

    IF report_text = '----- "' || temp_username || '" KULLANICISININ AKTİVİTE RAPORU -----' || E'\n\n' THEN
        RETURN 'Henüz bir aktivite kaydınız bulunmuyor.';
    ELSE
        RETURN report_text;
    END IF;
END;
$$ LANGUAGE plpgsql;
```

```sql
-- ROTADAN GELEN PUANI HESAPLAMA FONKSİYONU --
CREATE OR REPLACE FUNCTION calculate_route_score(p_route_id INT)
RETURNS INT AS $$
DECLARE
    difficulty INT;
    steps INT;
    score INT;
BEGIN
    SELECT difficulty_level, avg_steps INTO difficulty, steps
    FROM routes WHERE route_id = p_route_id;

    score := (difficulty * 10) + (steps / 10); -- Puan = (Zorluk * 10) + (Adım / 100)

    RETURN score;
END;
$$ LANGUAGE plpgsql;
```

```sql
-- ETKİNLİKTEKİ KALAN KONTEJANI HESAPLAYAN FONKSİYON --
CREATE OR REPLACE FUNCTION get_remaining_quota(p_event_id INT)
RETURNS INT AS $$
DECLARE
    total_quota INT;
    used_quota INT;
BEGIN
    SELECT quota INTO total_quota FROM events WHERE event_id = p_event_id;
    SELECT COUNT(*) INTO used_quota FROM event_participants WHERE event_id = p_event_id;

    RETURN (total_quota - used_quota);
END;
$$ LANGUAGE plpgsql;
```

12. Madde: TRIGGER Kullanımı

```sql
-- PUAN DAĞITIMINI SAĞLAYAN TRIGGER --
CREATE OR REPLACE FUNCTION trg_award_points_func()
RETURNS TRIGGER AS $$
DECLARE
    points_to_award INT;
BEGIN
    IF NEW.status = 'Tamamlandı' AND OLD.status != 'Tamamlandı' THEN

        SELECT (calculate_route_score(e.route_id)) INTO points_to_award
        FROM events e
        WHERE e.event_id = NEW.event_id;

        IF points_to_award IS NULL THEN
            points_to_award := 0;
        END IF;

        UPDATE users SET total_points = total_points + points_to_award
        WHERE user_id = NEW.user_id;

        RAISE NOTICE 'Tebrikler! Etkinliği tamamladınız ve % puan kazandınız.', points_to_award;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_award_points
AFTER UPDATE ON event_participants
FOR EACH ROW
EXECUTE FUNCTION trg_award_points_func();
```

```sql
-- KONTENJAN KONTROLÜ SAĞLAYAN TRIGGER --
CREATE OR REPLACE FUNCTION trg_check_quota_func()
RETURNS TRIGGER AS $$
DECLARE
    remaining INT;
BEGIN
    remaining := get_remaining_quota(NEW.event_id);

    IF remaining <= 0 THEN
        RAISE EXCEPTION 'Üzgünüz, bu etkinlik için kontenjan dolmuştur!';
    ELSE
        RAISE NOTICE 'Kayıt işlemi başarılı. İyi eğlenceler!';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_quota
BEFORE INSERT ON event_participants
FOR EACH ROW
EXECUTE FUNCTION trg_check_quota_func();
```

13. Madde: 2 Farklı Kullanıcı Rolü

```sql
    role VARCHAR(20) NOT NULL CHECK (role IN ('admin', 'user')), -- Kullanıcı ROlü
```